

Strings - DeepDrive

Character: Any symbol that is readable by humans, "a", "A", "1", . , kisi bhi language ka alphabet.

String: It is sequence of characters, it means it is the "chr []"--> character array.

- String is a non-primitive datatype.
- So string is a class, that comes under java.lang package.
- java.lang package is auto imported.

Properties of String class:

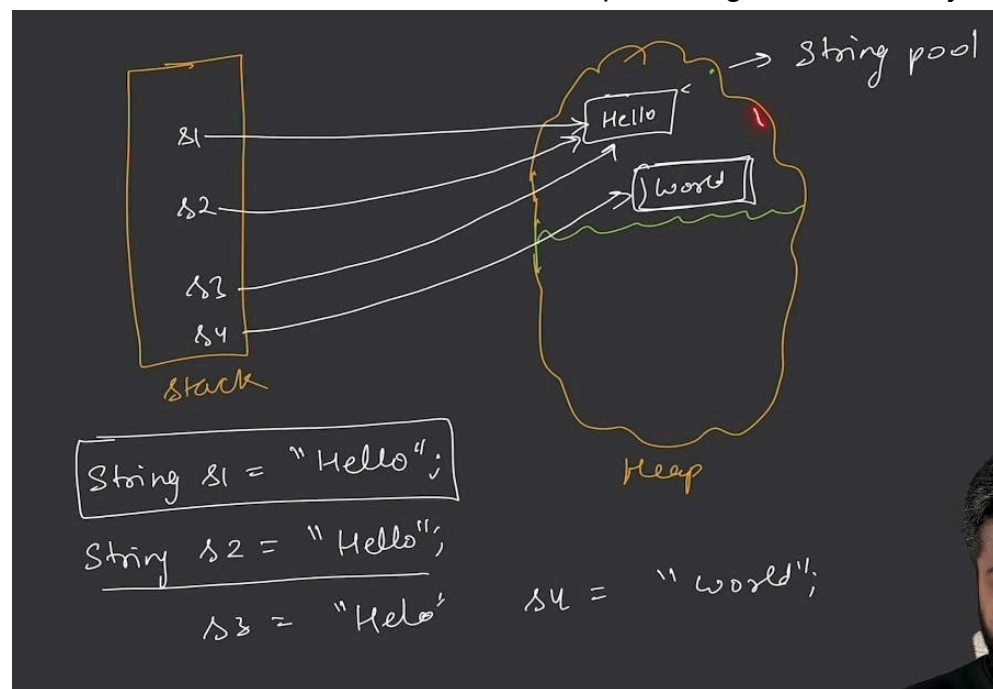
- String class is a **immutable** class, it means we can't change its any behaviour or data.
- Reason behind the immutable of class String is to be secure.

```
public class StringImmutable {  
    public static void main(String[] args) {  
        // String immutability check.  
        String s1 = "Hello";  
        s1.concat(" world");// too concat->string ke piche string lagana ke liye ha prr...  
        System.out.println(s1); //Result-->Hello-->ye concat string me possible nahi ha,  
        // kuki string immutable class ha ,  
        // orr "Hello world" alag object bn gya ha jo abhi khi point nahi ha.  
    }  
}
```

2 ways to declare String class objects→

1. **Using new operator:** iske literal ki copy bhi string pool me store hoti ha, bina kisi reference ke bhi.
2. **Using literal**→ "Priyansh" it unlocks a new feature of java "STRING POOL".

String Pool: ye heap memory me alog se ek jagah ha jisme literal objects store hote ha, or agar koi reference ka same literal ha to wo same hi literal ko point kreghe na ki new object bnayega.

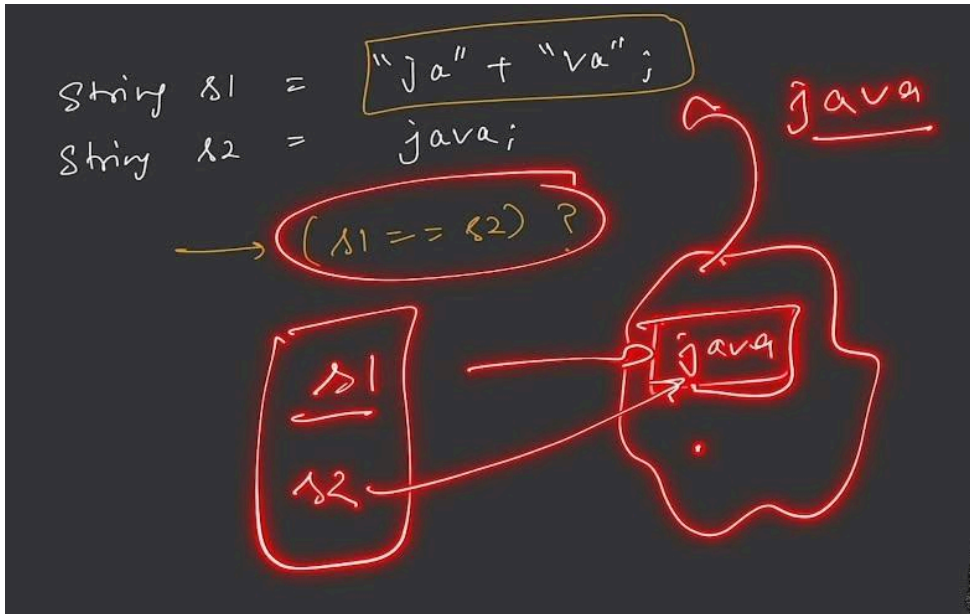


- It means ki hum strings ko reuse krte ha.
- This is part of heap memory in modern java but,
- It was not part of heap memory before,
- There was a special memory (PermGen) where String pools were stored.

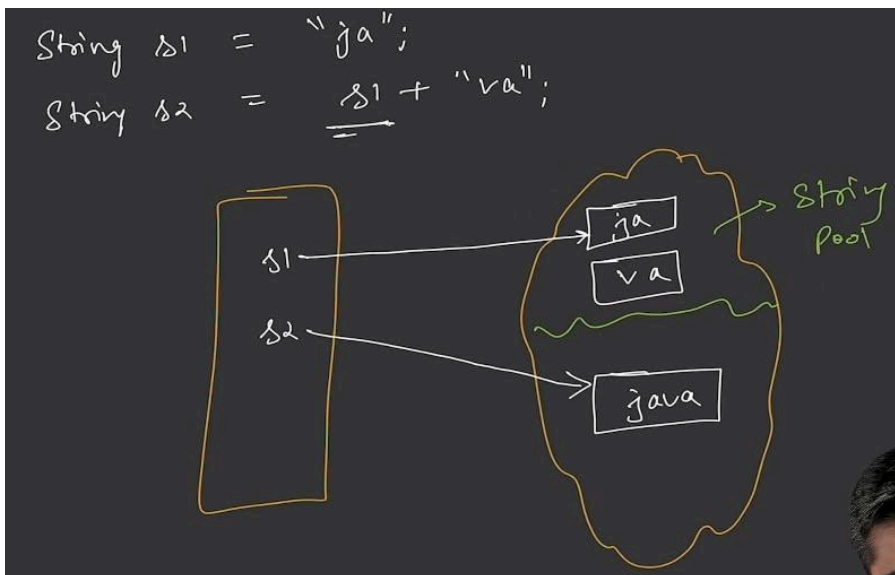
```
public class StringPool {  
    public static void main(String[] args) {  
        String s1 = "Hello"; //literals objects  
        String s2 = "Hello";  
        System.out.println(s1==s2); //True-->kuki dono ek hi object ko reffer kr rhe ha.  
        String s3 = new String ("Priyansh");  
        String s4 = new String ("Priyansh");  
        System.out.println(s3==s4); // false--> kuki dono alag-alag objects ko reffer kr rhe ha.  
    }  
}
```

Golden Rules: for String pool.

- Only compile time constants go to String pool automatically.



- Runtime created String go to heap.



```
public class GoldenStringRule {
    public static void main(String[] args) {
        String s1 = "ja" + "va"; // compile-time pr ho ta ha. StringPool memory me bnegha.
        String s2 = "java";
        System.out.println(s1==s2); // True--> same object ko reference kr rhe ha
        String s3 = "ja";
        String s4 = s3 + "va"; // run-time pr ho rha ha. "s3" heap memory me bnegha
        System.out.println(s3==s4); // false--> alag objects ko refer kr rhe ha.
        String s5 = "java"; // compile-time pr hi ho ri ha kuki assignment operator ha isme.
        String s6 = s5;
        System.out.println(s5==s6); // true --> pointing same reference in Stringpool.
        String s = "Hello"; // String pool me store hoga.
        s = "World"; // s ab world ko point krega.
        System.out.println(s); // World is answer.
    }
}
```

Problem of immutability: it wastes alot of memory.

```
public class ImmutabilityProblem {
    public static void main(String[] args) {
        // Problem of immutability.
        String s = ""; // /only this will store in string pool.
        for (int i=0; i<5; i++) {
```

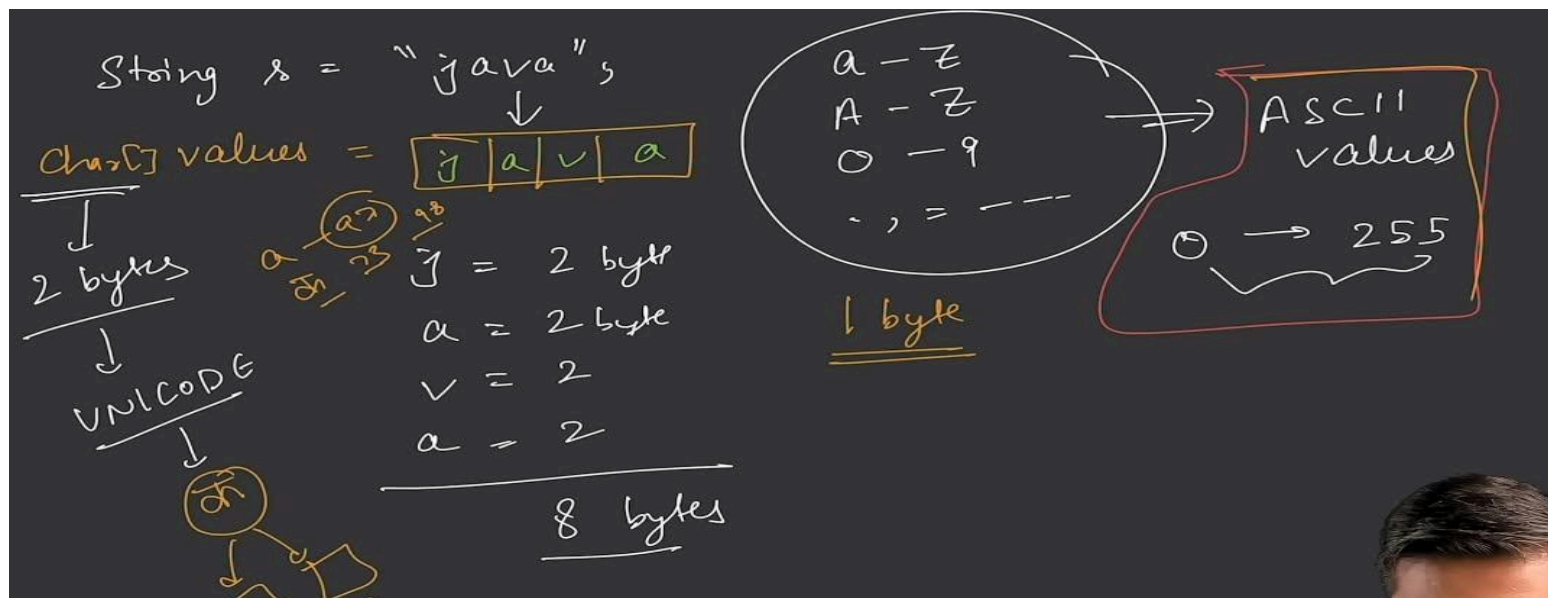
```
s+=i;
```

```
System.out.println(s); // "0"->"01"->"012"->"0123"->"01234", these all will store in heap memory separately.
```

- Java 9 se phle String class me internally values char[] array me store hoti thi.
- Ab values byte[] array me store hoti ha.

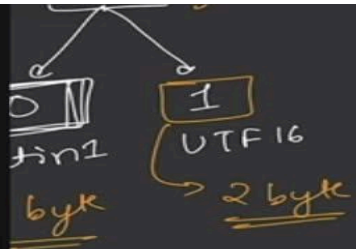
Reason→

- String me commonly use hone wale alphabets, symbols, etc ASCII-values ke hote ha.
- Or standard ASCII table me 128 characters hote ha,
- Or 1 byte total 256 alag-alag values ko store kr skta ha.
- Isliye ASCII ka koi bhi chracter 1byte me fit ho jata ha,
- Or 1 char ke ander 2bytes hoti jo total 65,536 alag-alag values ko store kr sktal ha.
- Or hume itni faltu storage waste krne ki zarurat nhi ha.
- Tabhi string values ko java 9 ke baad char[] se byte[] me change kr diya.



Byte Coder:

- Iski sirf 2 hi values hoti ha:--> 0, 1.
- Byte coder string class ko ye batati ha ki agar value 0 ha to ek bar me ek 1 character hi read krna , or agar value 1 ha to ek bar me 2 characters ko read krna.
- 0 value tab hoti ha jab string ke ander ASCII characters ki values dalne wale hote ho, or iska naam diya ja **LATIN9** format, ye leta ha **1byte character**.
- 1 value tab hoti ha jab string ke ander ASCII characters ki limit ke bhar jate ho, iska naam ha **UTF16** format, ye leta ha **2bytes characters**.



UNICODE

s = "java"

values[] = [74, 97, 118, 97]

coder = 0

s = "hi";

values[] = [?, ?]

coder = 1;

